```
  <ul>{items.map(renderItem)}</ul>
);
// Usage of the generic List component
interface User {
  id: number;
  name: string;
}
const users: User[] = [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Bob' }
];
const renderUser = (user: User) => <li key={user.id}>{user.name}<
<List<User> items={users} renderItem={renderUser} /> #E
```

NOTE

The syntax <T,> is a TypeScript/JSX workaround that uses a trailing comma to distinguish generic type parameters from JSX tags in .tsx files, avoiding ambiguity for the parser.

Constraints restrict the types that can be used as arguments by requiring them to meet specific criteria. The `extends` keyword specifies that a type must have certain properties or implement a particular interface. In listing 3.16, `<T extends User>` ensures that T must be compatible with the `User` interface or type.

Listing 3.16 Interface constraint

```
interface User {
  id: number;
  name: string;
}
function getUserInfo<T extends User>(user: T): string { #A
  return `ID: ${user.id}, Name: ${user.name}`;
}
```

Specific properties can be required for a type. For instance, a function that only accepts types with a `.length` property may be written as listing 3.17.

Listing 3.17 Property constraint

```
interface Lengthwise {
```